

Aplicación Web en PHP con NGINX

Elena Cusac Ortigosa



Automatización y despliegue de arquitectura

2ºDAW

Curso 2026-2027

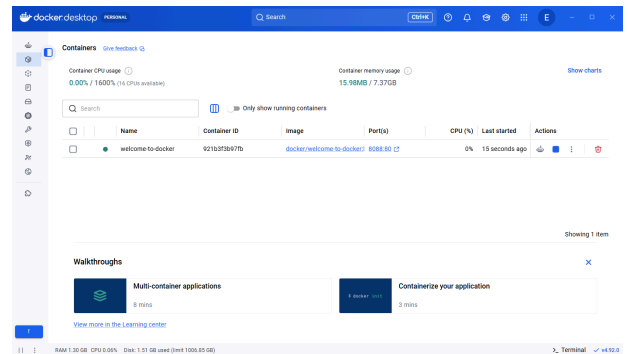
Índice

1. Instalación y configuración de Docker.....	3
2. Creación y preparación del Contenedor de PHP.....	3
3. Aplicación PHP.....	4

1. Instalación y configuración de Docker

Instalamos la última versión de Docker, en este caso para Windows 11, y mediante la interfaz iniciamos la ejecución de Docker.

Para esta tarea utilizaré WSL 2 para poder ejecutar un entorno de Linux dentro de mi terminal de Windows y realizarla de una manera más sencilla.



2. Creación y preparación del Contenedor de PHP

En la terminal de Ubuntu, con el comando `nano Dockerfile`, creo un nuevo Dockerfile con todas las instrucciones para copiar la imagen de PHP, instalar Nginx, indicar el puerto que utilizará y arrancar el contenedor.

```
elena@DESKTOP-VEE00N4: ~  
GNU nano 8.7.1 Dockerfile *  
FROM php:8.4-fpm  
  
RUN apt-get update \  
&& apt-get install -y nginx \  
&& rm -rf /var/lib/apt/lists/*  
  
COPY nginx.conf /etc/nginx/nginx.conf  
  
COPY app/ /var/www/html/  
  
RUN chown -R www-data:www-data /var/www/html  
  
EXPOSE 80  
  
CMD ["sh", "-c", "php-fpm -D && nginx -g 'daemon off;']
```

```
elena@DESKTOP-VEE00N4: ~  
GNU nano 8.7.1 nginx.conf *  
events {  
    worker_connections 1024;  
}  
  
http {  
    include /etc/nginx/mime.types;  
    default_type application/octet-stream;  
    sendfile on;  
    keepalive_timeout 65;  
    server {  
        listen 80;  
        server_name localhost;  
        root /var/www/html;  
        index index.php index.html;  
        location / {  
            try_files $uri $uri/ /index.php?$query_string;  
        }  
        location ~ \.php$ {  
            include fastcgi_params;  
            fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;  
            fastcgi_pass 127.0.0.1:9000;  
        }  
    }  
}
```

Una vez creado el *Dockerfile*, creamos el archivo de configuración de Nginx para que este sepa que los archivos `.php` tienen que ser procesados por PHP-FPM.

Aplicación Web en PHP con NGINX

Elena Cusac Ortigosa

Con el comando *docker build -t practica-php* creo la imagen a partir del archivo *Dockerfile* que he creado anteriormente en el mismo directorio y le asigno el nombre *practica-php* a la imagen para poder usarla fácilmente.

```
elena@DESKTOP-VEE00NM4:~/practica-docker-php$ docker build -t practica-php .
[*] Building 25.7s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 327B
=> [internal] load metadata for docker.io/library/php:8.4-fpm
=> [auth] library/php:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/php:8.4-fpm@sha256:223115611852f585846b5d30f0cd95de8af9b486ca3
=> => resolve docker.io/library/php:8.4-fpm@sha256:223115611852f585846b5d30f0cd95de8af9b486ca3
=> => sha256:84ff5d519c5f327d2ca55278904bcdc569ade358345e38c59ffbc7868e4c8f0 9.27kB / 9.27kB
=> => sha256:4f4f7b0ef34d61cfab2571ae8db9a8dc1e0cd55774b4a6d75e68dc38e8acc1 32B / 32B
=> => sha256:88076f60dad88eaa7a79373d6d894b5f9f5288014ca2a6a28a1cd76ad7f606 243B / 243B
=> => sha256:3646cb5170ee2b5b1d663f36a6958a85a222caff95fac7f630e8b3d9e4d7e12 249B / 249B
=> => sha256:399c9f64ad923f6eef4d97dceeb464804b16a6302b77d5c3b87980b3525368 2.45kB / 2.45kB
=> => sha256:d8ba7068ad1a62851cf7c38e7d23ff306767a9071831faac587f2e838e6020a 13.82MB / 13.82MB
=> => sha256:92998e6061c8f888ed57cbeb1b6a9d7b6f0a3d682b539f69cac52f769286a36 488B / 488B
=> => sha256:b7b01c2af1c86abace8639ac6cd277105704b1ec3f8a43c35816c27e3753cd 13.91MB / 13.91MB
=> => sha256:f5d4a83d9959130e2f66db1cb0c952falc42badc1eb3729a35c10f2b0dc2704 226B / 226B
```



Inicio el contenedor mediante *Docker run*, indicando que el contenedor se ejecute en segundo plano, el puerto de mi ordenador y el puerto del servidor web, en este caso Nginx.

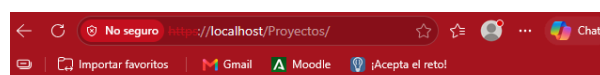
Pruebo localmente en el navegador accediendo al puerto si mi aplicación PHP funciona correctamente con el contenedor de Docker.

3. Aplicación PHP

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Práctica Docker PHP</title>
7 </head>
8 <body>
9
10  <h1>Aplicación PHP funcionando con Docker</h1>
11
12  <p>Esta página está siendo ejecutada dentro de un contenedor Docker.</p>
13
14  <p><strong>Servidor:</strong> Nginx</p>
15
16  <p><strong>Lenguaje:</strong> PHP</p>
17
18  <p><strong>Entorno:</strong> WSL2 + Docker</p>
19
20  <h2>Información de PHP</h2>
21
22  <?php
23    echo "<p>PHP está funcionando correctamente.</p>";
24    echo "<p>Versión de PHP: " . phpversion() . "</p>";
25    echo "<p>Fecha y hora del servidor: " . date("d/m/Y H:i:s") . "</p>";
26  ?>
27
28 </body>
29 </html>
```

Con Xampp compruebo localmente que el archivo se ejecuta correctamente y como debería.

Creo un archivo PHP llamado *index.php* en el que estará mi aplicación web. Al final del código creo una sección PHP que muestre la versión de PHP que se está ejecutando y la fecha y hora a la que se ha ejecutado el script para comprobar que todo funciona correctamente



Aplicación PHP funcionando con Docker

Esta página está siendo ejecutada dentro de un contenedor Docker.

Servidor: Nginx

Lenguaje: PHP

Entorno: WSL2 + Docker

Información de PHP

PHP está funcionando correctamente.

Versión de PHP: 8.2.12

Fecha y hora del servidor: 24/09/2026 16:41:14